

Node and Graph Classification

Peter Dolog

dolog@cs.aau.dk

<http://people.cs.aau.dk/~dolog>

Slides based on chapter 5 from graph representation learning book.
Some information also from chapter Node Classification in Social Network

Outline

Transformer Architecture

Graph Embedding as Node Embedding Aggregation

Node and Graph Classification

MULTI-HEAD ATTENTION AND RELATION TO TRANSFORMER ARCHITECTURE

Neighborhood Attention

Assigning an attention/importance weight for each neighbor
Weighing the influence of each neighbor in the aggregation

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h}_v$$

$$\alpha_{u,v} = \frac{\exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} \exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}])}$$

a ... learnable attention vector

W ... learnable matrix

$\oplus$... concatenation

Alternative Attention Computation

$$\alpha_{u,v} = \frac{\exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} \exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_{v'}])}$$

OR

$$\alpha_{u,v} = \frac{\exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_v))}{\sum_{v' \in \mathcal{N}(u)} \exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_{v'}))}$$

Idea

Consider an idea that you want to learn also dependencies and weights of nodes with respect to all other nodes

Similar idea appeared in sequential language models also w.r.t. context

Context is another concept which is added to the model in a form of query

Given a query, there is a sequential dependency on some words but not all

Attention is good for scoring that

But we need multiple attentions for different contexts

This is solved by Multi-Head Attention

Similar situation for graphs

Neighborhood Attention

Assigning an attention/importance weight for each neighbor
Weighing the influence of each neighbor in the aggregation

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h}_v$$

$$\alpha_{u,v} = \frac{\exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} \exp(\mathbf{a}^T [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}])}$$

a ... learnable attention vector

W ... learnable matrix

$\oplus$... concatenation

Alternative Attention Computation

$$\alpha_{u,v} = \frac{\exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_v])}{\sum_{v' \in \mathcal{N}(u)} \exp([\mathbf{h}_u^T \mathbf{W} \mathbf{h}_{v'}])}$$

OR

$$\alpha_{u,v} = \frac{\exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_v))}{\sum_{v' \in \mathcal{N}(u)} \exp(\mathbf{MLP}(\mathbf{h}_u, \mathbf{h}_{v'}))}$$

Multi-Head Attention

$$\mathbf{m}_{\mathcal{N}(u)} = [\mathbf{a}_1 \oplus \mathbf{a}_2 \oplus \cdots \oplus \mathbf{a}_K]$$

$$\mathbf{a}_k = \mathbf{W}_k \sum_{v \in \mathcal{N}(u)} \alpha_{u,v,k} \mathbf{h}_v$$

Multi-Head Attention

$$\mathbf{m}_{\mathcal{N}(u)} = [\mathbf{a}_1 \oplus \mathbf{a}_2 \oplus \cdots \oplus \mathbf{a}_K]$$

$$\mathbf{a}_k = \mathbf{W}_k \sum_{v \in \mathcal{N}(u)} \alpha_{u,v,k} \mathbf{h}_v$$

Computed by one of the previously mentioned approaches for single attention weight

Transformer Architecture

(C) Attention is All you need paper
<https://arxiv.org/abs/1706.03762>

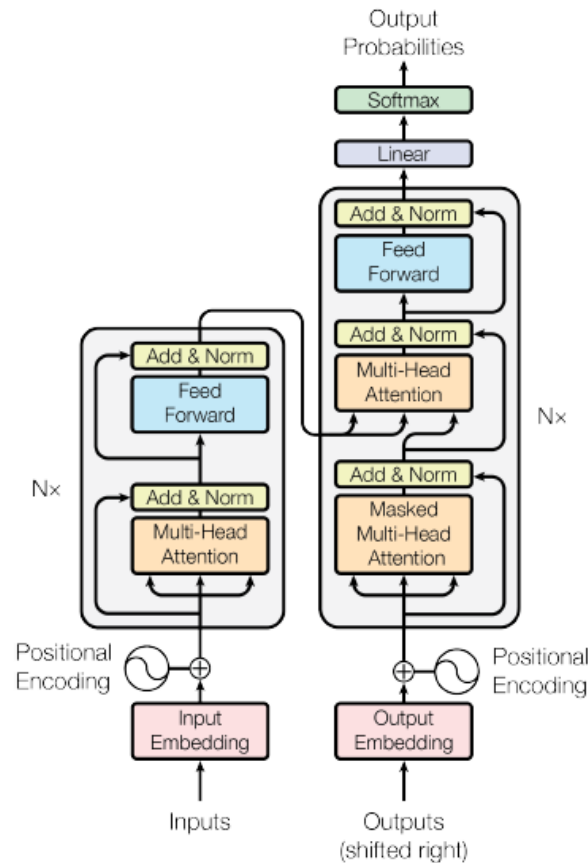


Figure 1: The Transformer - model architecture.

Transformer Architecture

(C) Attention is All you need paper
<https://arxiv.org/abs/1706.03762>

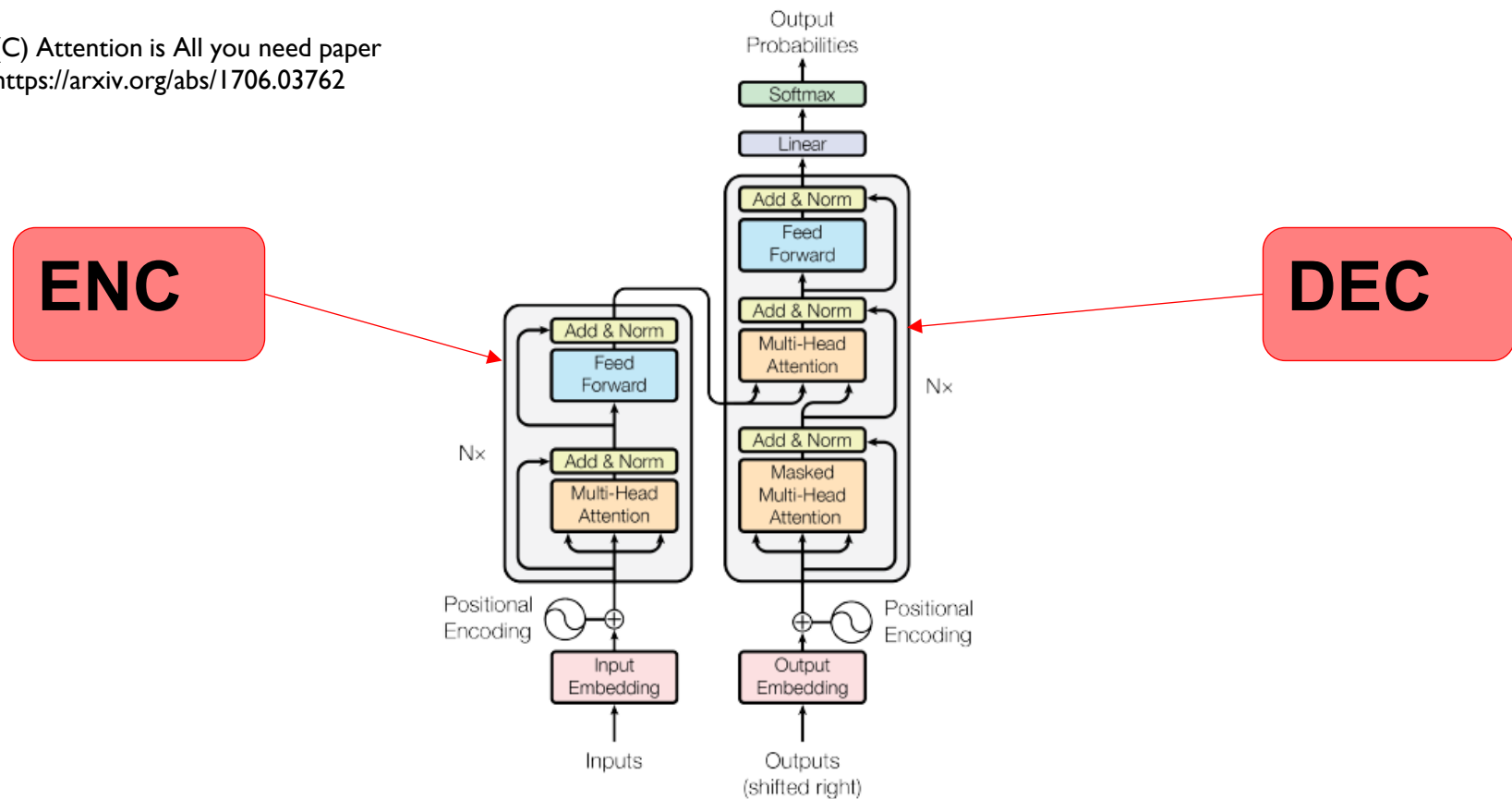


Figure 1: The Transformer - model architecture.

Transformer Architecture

(C) Attention is All you need paper
<https://arxiv.org/abs/1706.03762>

ENC

**Basic component:
Multi-Head
Attention
followed by
LIN
(normalized)**

DEC

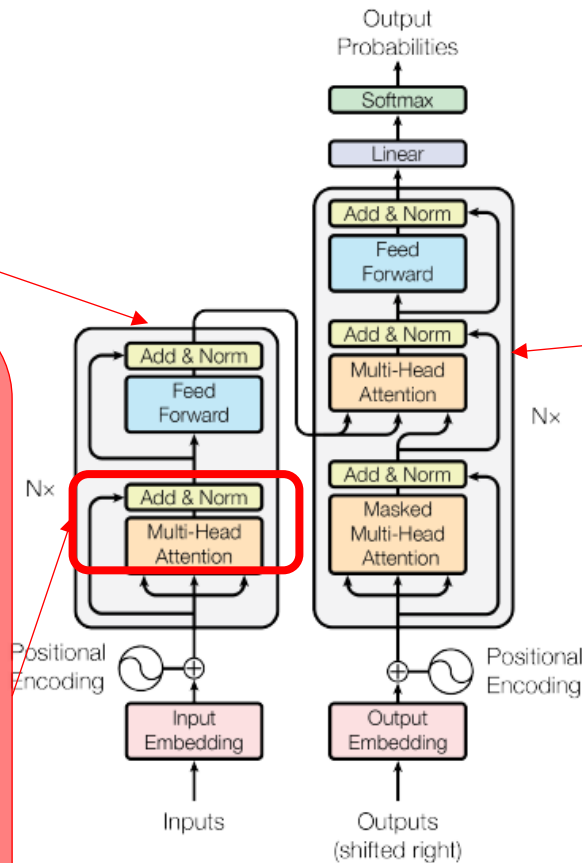


Figure 1: The Transformer - model architecture.

Advantages

If we have some prior knowledge about importance
Importance differ for different situations for example different
label for classification

AGGREGATION FOR GRAPH EMBEDDINGS

Graph Pooling

Sum or Mean of Node Embeddings (often sufficient esp. for small graphs):

$$\mathbf{z}_G = \frac{\sum_{u \in \mathcal{V}} \mathbf{z}_u}{f_n(|\mathcal{V}|)}$$

Combination of LSTMs and attention:

$$\begin{aligned}\mathbf{q}_t &= LSTM(\mathbf{o}_{t-1}, \mathbf{q}_{t-1}), \\ e_{v,t} &= f_a(\mathbf{z}_v, \mathbf{q}_t), \forall v \in \mathcal{V}, \\ a_{v,t} &= \frac{\exp(e_{v,t})}{\sum_{u \in \mathcal{V}} \exp(e_{u,t})} \forall v \in \mathcal{V}, \\ \mathbf{o}_t &= \sum_{v \in \mathcal{V}} a_{v,t} \mathbf{z}_v \\ \mathbf{z}_G &= \mathbf{o}_1 \oplus \mathbf{o}_2 \oplus \dots \oplus \mathbf{o}_T\end{aligned}$$

Graph Pooling

Sum or Mean of Node Embeddings (often sufficient esp. for small graphs):

$$\mathbf{z}_G = \frac{\sum_{u \in \mathcal{V}} \mathbf{z}_u}{f_n(|\mathcal{V}|)}$$

Combination of LSTMs and **attention**:

$$\begin{aligned}\mathbf{q}_t &= LSTM(\mathbf{o}_{t-1}, \mathbf{q}_{t-1}), \\ e_{v,t} &= f_a(\mathbf{z}_v, \mathbf{q}_t), \forall v \in \mathcal{V}, \\ a_{v,t} &= \frac{\exp(e_{v,t})}{\sum_{u \in \mathcal{V}} \exp(e_{u,t})} \forall v \in \mathcal{V}, \\ \mathbf{o}_t &= \sum_{v \in \mathcal{V}} a_{v,t} \mathbf{z}_v \\ \mathbf{z}_G &= \mathbf{o}_1 \oplus \mathbf{o}_2 \oplus \dots \oplus \mathbf{o}_T\end{aligned}$$

Graph Pooling

Sum or Mean of Node Embeddings (often sufficient esp. for small graphs):

$$\mathbf{z}_G = \frac{\sum_{u \in \mathcal{V}} \mathbf{z}_u}{f_n(|\mathcal{V}|)}$$

Combination of LSTMs and attention:

$$\begin{aligned}\mathbf{q}_t &= LSTM(\mathbf{o}_{t-1}, \mathbf{q}_{t-1}), \\ e_{v,t} &= f_a(\mathbf{z}_v, \mathbf{q}_t), \forall v \in \mathcal{V}, \\ a_{v,t} &= \frac{\exp(e_{v,t})}{\sum_{u \in \mathcal{V}} \exp(e_{u,t})} \forall v \in \mathcal{V}, \\ \mathbf{o}_t &= \sum_{v \in \mathcal{V}} a_{v,t} \mathbf{z}_v\end{aligned}$$

**Graph
Embedding**

$$\mathbf{z}_G = \mathbf{o}_1 \oplus \mathbf{o}_2 \oplus \dots \oplus \mathbf{o}_T$$

Graph Coarsening

Applying a clustering:

$$f_c \rightarrow \mathcal{G} \times \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^{+|V| \times c}$$

Creating clustering assignment matrix:

$$\mathbf{S} = f_c(\mathcal{G}, \mathbf{Z}), \mathbf{S}[u, i] \in \mathbb{R}^+$$

Such matrix is used

- to coarsen the graph:

$$\mathbf{A}^{new} = \mathbf{S}^T \mathbf{A} \mathbf{S} \in \mathbb{R}^{+c \times c}$$

- compute a new set of features:

$$\mathbf{X}^{new} = \mathbf{S}^T \mathbf{X} \in \mathbb{R}^{+c \times d}$$

Run for multiple interactions where number of clusters is decreased continuously

NODE AND GRAPH CLASSIFICATION

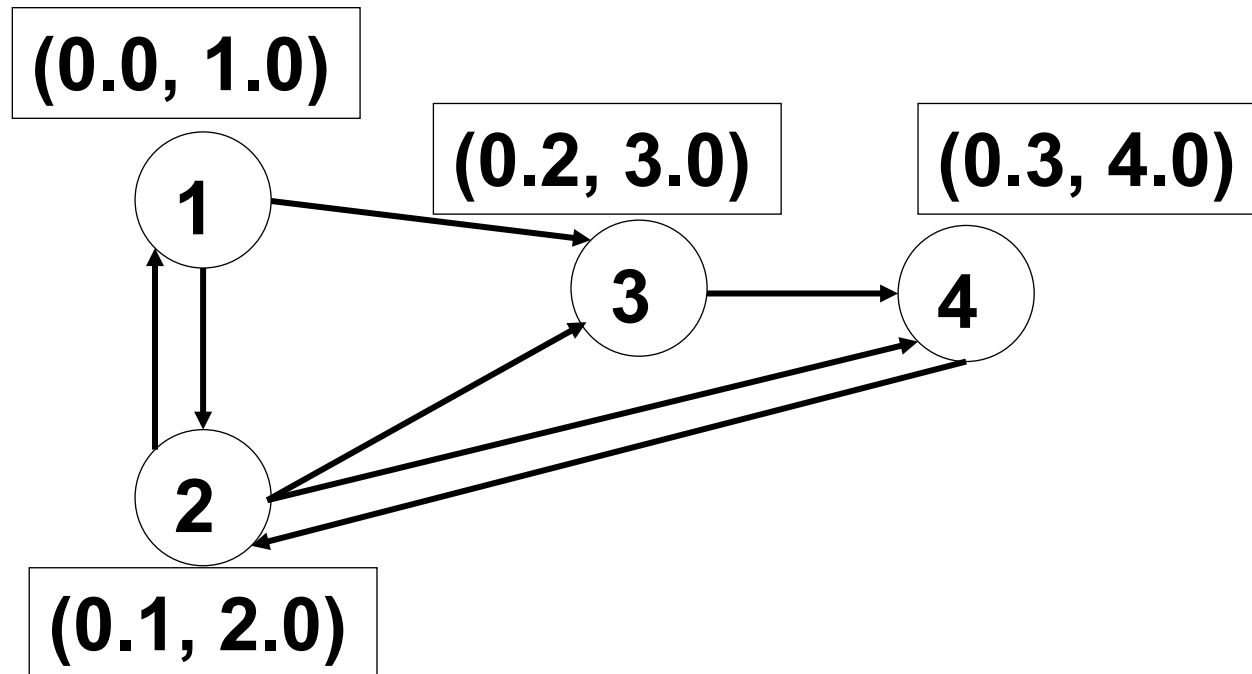
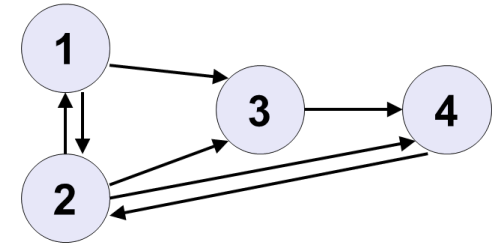
Node Classification

Given the node features or node neighbors, predict label/class for a node

2 situations:

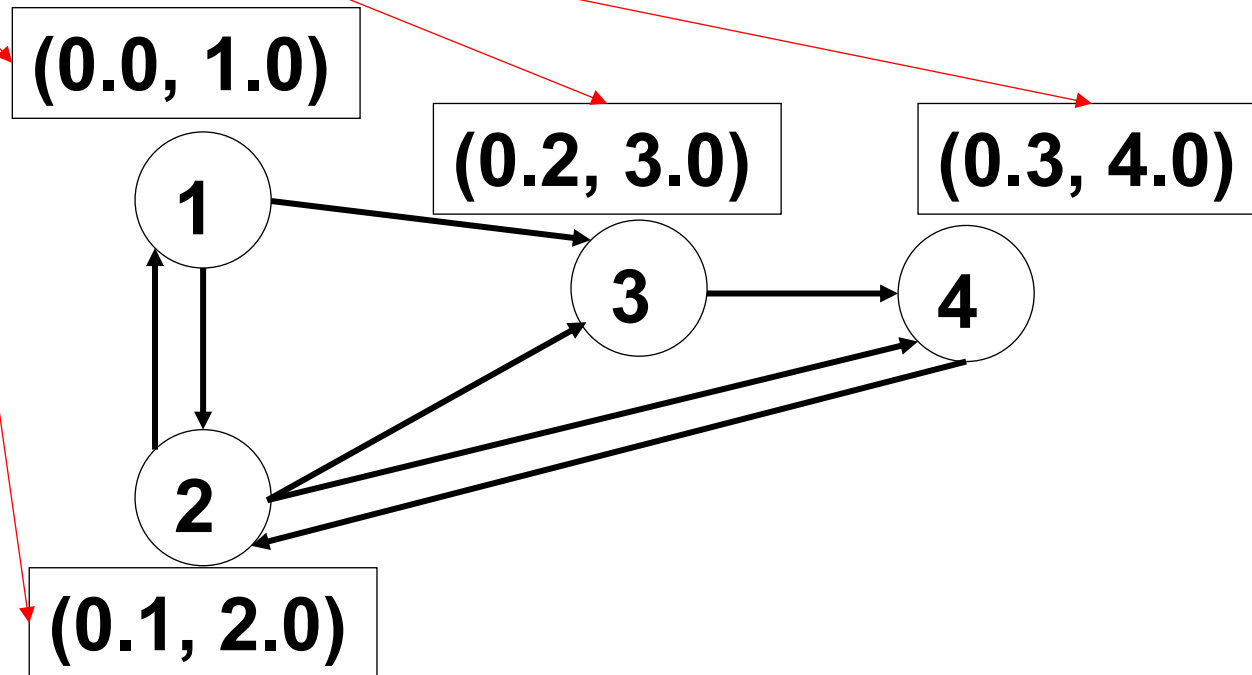
- I have some "content" data for the nodes, graph structure and labels for some of the nodes
- I only have a graph structure and labels for some of the nodes

Illustration

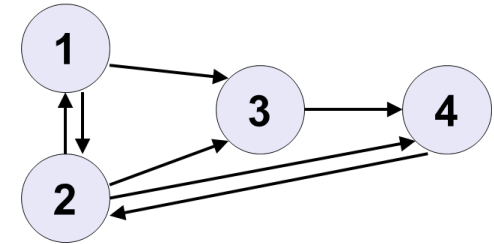


Illustration

Features



Illustration

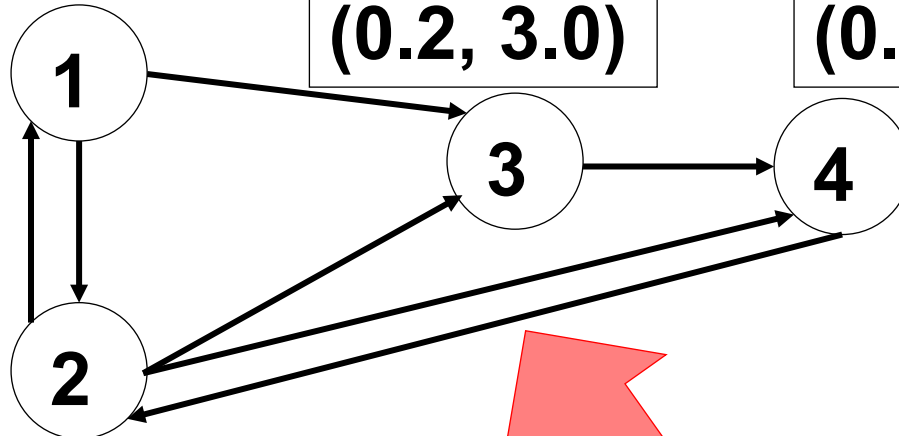


Features

(0.0, 1.0)

(0.2, 3.0)

(0.3, 4.0)



(0.1, 2.0)

Graph Structure

Illustration

Features

Label2

Label1

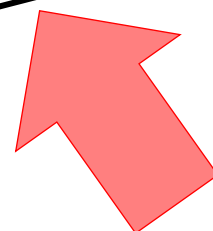
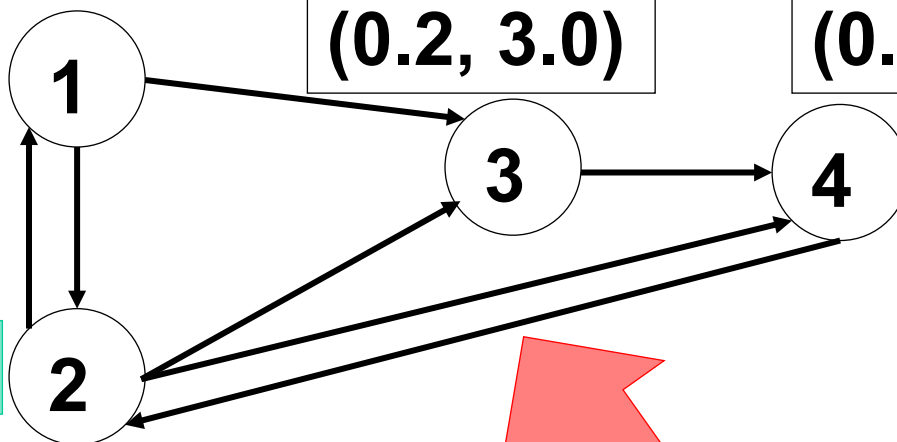
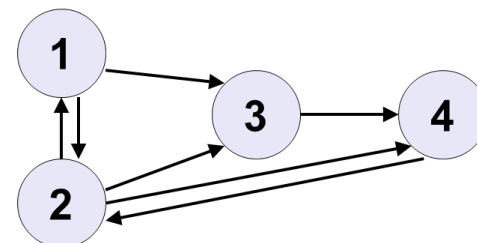
Graph Structure

(0.0, 1.0)

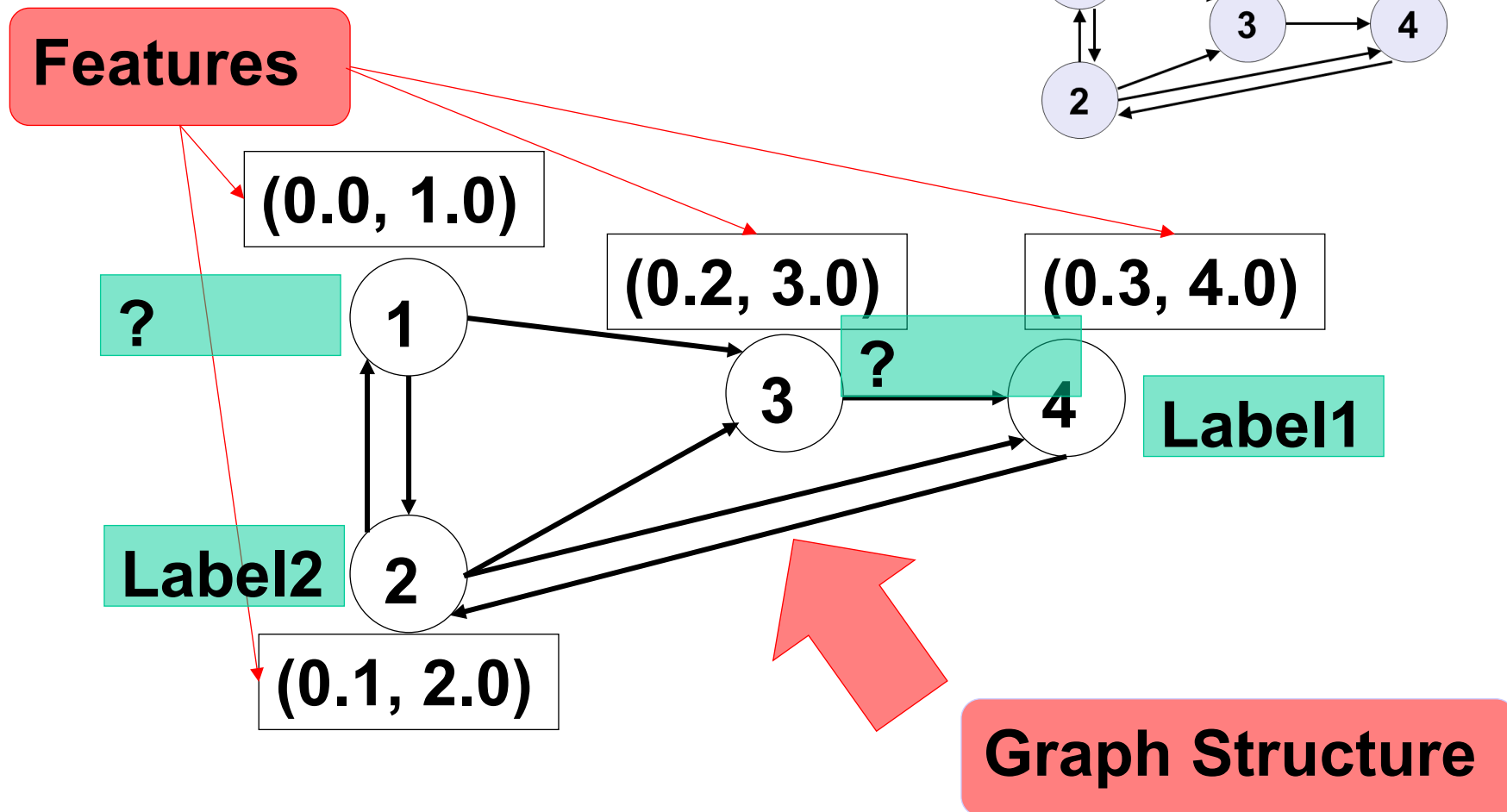
(0.2, 3.0)

(0.3, 4.0)

(0.1, 2.0)



Illustration



Classification from LI

For all data points we have:

- Known values for a set of features (a.k.a. predictors, attributes, ...)
- A known or unknown label

The difference:

- Graph structure

Can we still reuse the knowledge from the LI or do we need to go beyond?

Classification from LI

For all data points we have:

- Known values for a set of features (a.k.a. predictors, attributes, ...)
- A known or unknown label

The difference:

- Graph structure

Can we still reuse the knowledge from the LI or do we need to go beyond?

To certain extent yes....

Classification from L1 and from the course until now

L1:

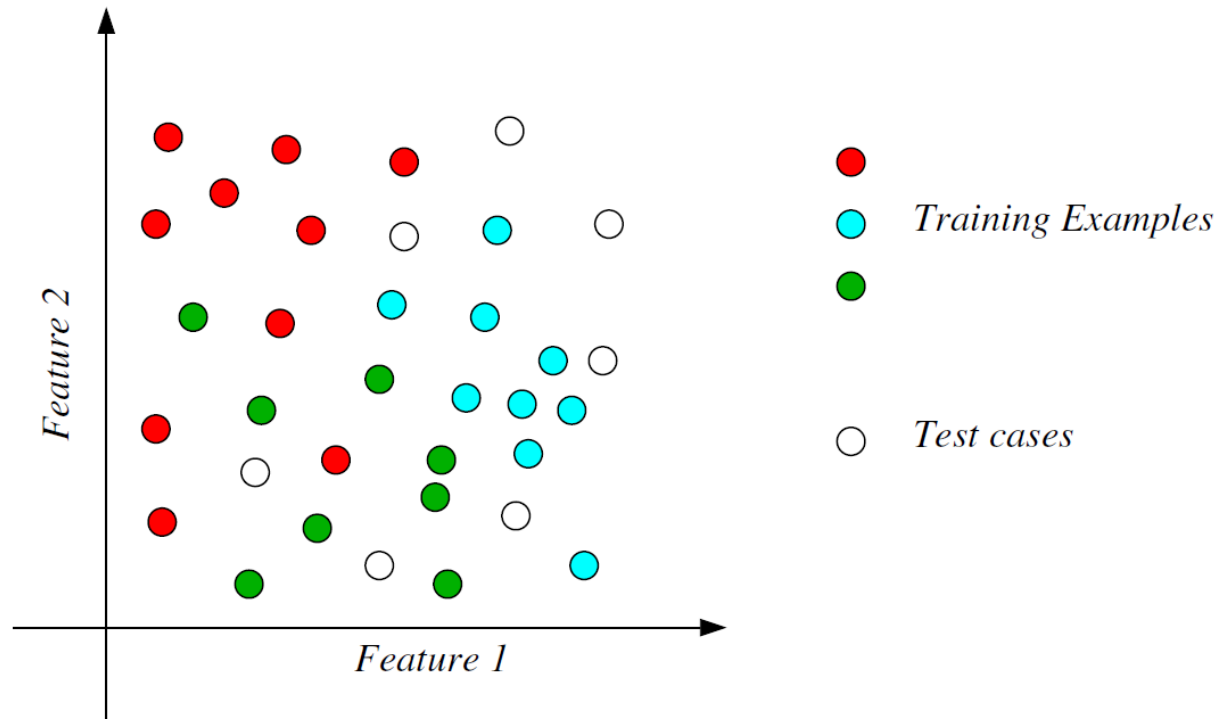
- K-Nearest Neighbor
- Perceptron

L2:

- Probabilistic Models
- Data Transformation

...

K-Nearest Neighbor from LI



Near neighbors tend to have the same labels

K-Nearest Neighbor from LI

Data: Set of labeled training instances $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$ consisting of D -dimensional input feature (attribute) vectors $\mathbf{x}_i = (x_{i,1}, \dots, x_{i,D})$ and a class label y_i

Required: distance function $d(\mathbf{x}, \mathbf{x}')$ to measure distances between attribute vectors (e.g. Euclidean Distance if all attributes are real numbers, i.e $\mathbf{x} \in \mathbb{R}^D$).

Classify new instance $(\mathbf{x}, ?)$ as follows:

- Let $(\mathbf{x}_{j,1}, y_{j,1}), \dots, (\mathbf{x}_{j,K}, y_{j,K})$ be K training instances whose attributes are closest to $\mathbf{x}$.
- Predict for $(\mathbf{x}, ?)$ the class label that occurs the most frequently among $y_{j,1}, \dots, y_{j,K}$

K-Nearest Neighbor from LI

Data: Set of labeled training instances $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$ consisting of D-dimensional input feature (attribute) vectors $\mathbf{x}_i = (x_{i,1}, \dots, x_{i,D})$ and a class label y_i

Required: **distance function $d(\mathbf{x}, \mathbf{x}')$** to measure distances between attribute vectors (e.g. Euclidean Distance if all attributes are real numbers, i.e $\mathbf{x} \in \mathbb{R}^D$).

Classify new instance $(\mathbf{x}, ?)$ as follows:

- Let $(\mathbf{x}_{j,1}, y_{j,1}), \dots, (\mathbf{x}_{j,K}, y_{j,K})$ be K training instances whose attributes **are closest to $\mathbf{x}$** .
- Predict for $(\mathbf{x}, ?)$ the class label that occurs the most frequently among $y_{j,1}, \dots, y_{j,K}$

L8-I0

Distance is related to Similarity

More distant, less similar

L8: Structural similarity on graphs – local similarity, random walk-based similarity

L9:

- Similarity on an embedding space – graph transformation into coordinates and similarity – cosine and Euclidean
- Reconstructing based on graph structure or a random walk

L10:

- Similarity on an embedding space
- Feature transformation with graph structure information

Classification as KNN with Graphs

Classify new instance $(\mathbf{x}, ?)$ as follows:

- Let $(\mathbf{x}_{j,1}, y_{j,1}), \dots, (\mathbf{x}_{j,K}, y_{j,K})$ be K training instances whose attributes **are SIMILAR to $\mathbf{x}$. (either on graph or embedding space)**
- Predict for $(\mathbf{x}, ?)$ the class label that occurs the most frequently among $y_{j,1}, \dots, y_{j,K}$

Note: similarity defined between objects based on the objects characteristics can serve as a constructing factor for a graph, e.g. similarity over certain threshold can lead to a link – which is exactly what we had in the link prediction scenario

Classification as Naïve Bayes or Perceptron (LI)

Embeddings space constructed by node and graph embedding methods provides a feature space

Any classification method on top of such embeddings can be applied

Concepts from Node Classification

Homophily (birds of a feather): link between individuals is correlated with similarity in characteristics of those individuals

Co-citation regularity: similar individuals tend to refer or connect to the same thing

Iterative Classification Method

Iteratively applying classifier

Algorithm 1: ICA(V, E, W, Y_l)

Compute Φ^1 from V, E, W, Y_l

Train classifier using Φ_l

for $t \leftarrow 1$ to τ do

 Apply classifier to Φ_u^t to compute Y_u^t
 Update Φ_u^t

$\tilde{Y} \leftarrow Y^\tau$

return $\tilde{Y}$

(C) Chapter Node Classification in Social Networks

V, E, W, Y_l are nodes, edges, weights and node labels in the graph respectively

Φ denotes matrix of feature vectors

$Y_u^t, Y^\tau, \tilde{Y}$ are computed labels based on classifier

Iterative Classification Method

Iteratively applying classifier

Algorithm 1: ICA(V, E, W, Y_l)

Compute Φ^1 from V, E, W, Y_l

Train classifier using Φ_l

for $t \leftarrow 1$ to τ do

 Apply classifier to Φ_u^t to compute Y_u^t

 Update Φ_u^t

$\tilde{Y} \leftarrow Y^\tau$

return $\tilde{Y}$

**Depending on
method of choice,
we aggregate
neighborhood
features or labels**

(C) Chapter Node Classification in Social Networks

V, E, W, Y_l are nodes, edges, weights and node labels in the graph respectively

Φ denotes matrix of feature vectors

$Y_u^t, Y^\tau, \tilde{Y}$ are computed labels based on classifier

Illustration

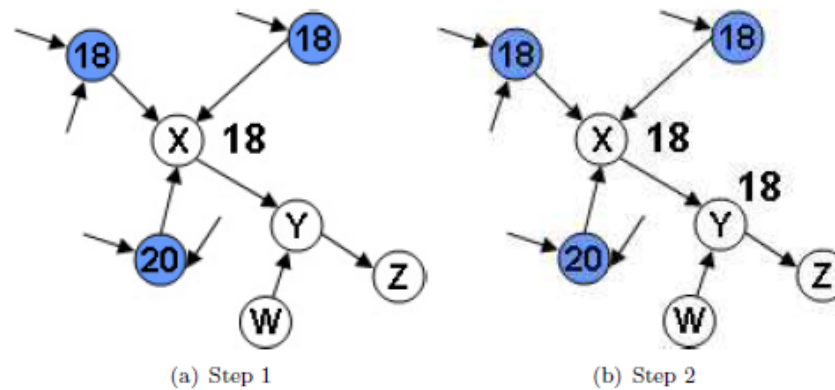


Figure 1.1. Two steps of a local iterative approach to node classification

(C) Chapter Node Classification in Social Networks

WHY IT WORKS

Random Walk Based Methods

Idea:

$$\tilde{y}_i[c] = \sum_{j|v_j \in V_l} p_{ij}^{\infty} y_j[c]$$

Total probability of reaching nodes labeled c on convergence
starting at v_i

Matrix formulation:

$$\tilde{Y} = P^{\infty} Y$$

Y is a matrix with probability distribution for labeled nodes and 0
for unlabeled

Random Walk Based Methods

Idea:

$$\tilde{y}_i[c] = \sum_{j|v_j \in V_l} p_{ij}^{\infty} y_j[c]$$

Total probability of reaching nodes labeled c on convergence
starting at v_i

Matrix formulation:

$$\tilde{Y} = P^{\infty} Y$$

Y is a matrix with probability distribution for labeled nodes and 0 for unlabeled

Label Propagation with Random Walk

If a node is labeled, outgoing transition does not change the labeling => the node loops to itself, random walk stops

If a node is unlabeled – transition proportional to the edge weight

The labeled nodes are ordered before the unlabeled ones

$$P = \begin{pmatrix} P_{ll} & P_{lu} \\ P_{ul} & P_{uu} \end{pmatrix} = \begin{pmatrix} I & 0 \\ P_{ul} & P_{uu} \end{pmatrix}$$

Recall Random Walk

The way we **compute** transitions:

$$q_j^{(t)} = \sum_{i=1}^n q_i^{(t-1)} P_{i,j} = \sum_{i:i \rightarrow j}^n q_i^{(t-1)} P_{i,j}$$

$P_{i,j} = 0$ when $i \nrightarrow j$

computed for all pages $j \in \{1, \dots, n\}$

Matrix notation, the way we **talk** about it:

$$\mathbf{q}^{(t)} = \mathbf{q}^{(t-1)} \mathbf{P}$$

$$\begin{pmatrix} q_1^{(t)} & \dots & q_j^{(t)} & \dots & q_n^{(t)} \end{pmatrix} = \begin{pmatrix} q_1^{(t-1)} & \dots & q_j^{(t-1)} & \dots & q_n^{(t-1)} \end{pmatrix} \begin{pmatrix} P_{1,1} & \dots & P_{1,j} & \dots & P_{1,n} \\ \dots & \dots & \dots & \dots & \dots \\ P_{i,1} & \dots & P_{i,j} & \dots & P_{i,n} \\ \dots & \dots & \dots & \dots & \dots \\ P_{n,1} & \dots & P_{n,j} & \dots & P_{n,n} \end{pmatrix}$$

What does it mean for us

We need random walks for each node, i.e. starting at each node and running until convergence

$$e_{i,u}^{(0)} = \begin{cases} 1 & \text{at } i'\text{th position} \\ 0 & \text{anywhere else} \end{cases}$$

$$e_i^{(t)} = e_i^{(t-1)} P$$

e_i^∞ is a i' th row of P^∞

Label Propagation with Random Walk cntd.

$$P = \begin{pmatrix} P_{ll} & P_{lu} \\ P_{ul} & P_{uu} \end{pmatrix} = \begin{pmatrix} I & 0 \\ P_{ul} & P_{uu} \end{pmatrix}$$

**Does
not
change**

Label Propagation with Random Walk cntd.

$$P = \begin{pmatrix} P_{ll} & P_{lu} \\ P_{ul} & P_{uu} \end{pmatrix} = \begin{pmatrix} I & 0 \\ P_{ul} & P_{uu} \end{pmatrix}$$

**Does
not
change**

**P_{ul}^{∞} will be
distribution
of labeled
nodes**

For $P_{uu}^{\infty} = 0$

Label Propagation with Random Walk cntd.

$$P = \begin{pmatrix} P_{ll} & P_{lu} \\ P_{ul} & P_{uu} \end{pmatrix} = \begin{pmatrix} I & 0 \\ P_{ul} & P_{uu} \end{pmatrix}$$

Does not change

P_{ul}^{∞} will be distribution of labeled nodes

For $P_{uu}^{\infty} = 0$

$$P^{\infty} = \begin{pmatrix} I & 0 \\ (I - P_{uu})^{-1} P_{ul} & P_{uu}^{\infty} \end{pmatrix}$$

Label Propagation with Random Walk cntd.

$$P = \begin{pmatrix} P_{ll} & P_{lu} \\ P_{ul} & P_{uu} \end{pmatrix} = \begin{pmatrix} I & 0 \\ P_{ul} & P_{uu} \end{pmatrix}$$

Does not change

P_{ul}^{∞} will be distribution of labeled nodes

For $P_{uu}^{\infty} = 0$

$$P^{\infty} = \begin{pmatrix} I & 0 \\ (I - P_{uu})^{-1} P_{ul} & P_{uu}^{\infty} \end{pmatrix}$$

Solution for stationary distribution of random walk

Random Walk Based Methods

Idea:

$$\tilde{y}_i[c] = \sum_{j|v_j \in V_l} p_{ij}^{\infty} y_j[c]$$

Total probability of reaching nodes labeled c on convergence starting at v_i

Matrix formulation:

$$\tilde{Y} = P^{\infty} Y$$

Y is a matrix with probability distribution for labeled nodes and 0 for unlabeled

$$Y_u = P_{ul}^{\infty} Y_l = (I - P_{uu})^{-1} P_{ul} Y_l$$

Notes

If a node is labeled, outgoing transition does not change the labeling => the node loops to itself, random walk stops

- If we would not have this, the random walk converges to stationary distribution and does not matter where we start
- We could go around this by fixing number of steps to a small number – very sensitive
- Graph needs to be label connected

Algorithm

Algorithm 2: LP-ZHU(Y, P)

$Y^0 \leftarrow Y$

repeat

$Y^t \leftarrow PY^{t-1}$

$Y_l^t \leftarrow Y_l$

until *convergence to* Y^∞

$\tilde{Y} \leftarrow Y^\infty$

return $\tilde{Y}$

(C) Chapter Node Classification in Social Networks

Randevous approach to label propagation

If graph is label connected, there is 0 probability to arrive in convergence from one unlabeled node to another

Therefore:

$$P^{\infty} = \begin{pmatrix} I & 0 \\ P_{ul}^{\infty} & 0 \end{pmatrix}$$

It was shown that it is sufficient to compute l leading eigen vectors of P – eigen values of 1

$$P_{ij}^{\infty} = \frac{S_{ij}}{S_{jj}}$$

S -... eigen vectors

i, j ... indices unlabeled and labeled nodes respectively

Cheaper to compute than inverse of $(I - P_{uu})$

Adapted Algorithm

Algorithm 3: LP-ZHOU(Y, α, T)

```

 $t \leftarrow 1$ 
 $Y^0 \leftarrow Y$ 
repeat
   $Y^t \leftarrow \alpha T Y^{t-1} + (1 - \alpha) Y^0$ 
until convergence to  $Y^\infty$ 
foreach  $i \in V$  do
   $c = \arg \max_{j \in \mathcal{Y}} y_i^\infty[j]$ 
   $\tilde{y}_i[c] = 1$ 
return  $\tilde{Y}$ 

```

(C) Chapter Node Classification in Social Networks

The main point

We can boil it down to a dot product on the embedding space
between labels and learned representations
Further approaches exist for example for graph regularization
Those can be used to explain the main mechanics but also as
iterative methods

Graph Classification

If we have multiple graphs

We run node embedding methods for each graph

We aggregate graph embedding by one of the graph aggregations discussed in the beginning of this lecture

We run any of the classification algorithms on such embeddings to predict the labels

Conclusions

Additional thoughts on multi-head attention, transformers and graph aggregation

Node classification as a function on top of embeddings we have created

Both, random walk and standard similarity approaches apply

Also at the aggregated graph level for graph classification